Software Requirements Specification (SRS)

File Compression Tool — Huffman Coding

Item	Detail
Project	File Compression Tool
Document	Software Requirements Specification (SRS)
Version	2.0 — Updated
Date	6 November 2026
Format	IEEE 830 style (see §1.4)
Repository	`KeerthanaKumaresh1/file-compression-tool`
Author (repository / README)	Keerthana Kumaresh — GitHub username `KeerthanaKumaresh1`
Team (3 people, GitHub usernames)	`KeerthanaKumaresh1`, `rishi2908`, `khushi309`
Implementation	Python, Huffman coding, `.hfc` / `HFC1` file format
Interfaces	Command-line interface (CLI) and a 5-tab Tkinter GUI

> **A note on names.** Only one team member's full name is given in the evidence inspected: Keerthana Kumaresh, named as author in `README.md`. The other two team members are identified in this document by their GitHub usernames only (`rishi2908`, `khushi309`). Their full names were not in the evidence inspected and are not invented here — add them from the team list before submission if required.
>
> **A note on status.** This is the updated SRS required by the mini-project checklist and FAQs. It does not claim a pull request, a GitHub Actions run, or a demo video. Test execution results are not claimed in this SRS — they are in the Test Plan and Testing Reports delivered with it (81 tests, all passed).

Revision history

Version	Date	Author / change	Description
1.0	No date stated in the file	Team (see title page)	Original SRS in `documents/SRS_document.pdf`: a table of functional requirements FR-1 to FR-10 and non-functional requirements NFR-1 to NFR-8 only. No introduction, product description, use-case diagram, security objectives, traceability, or file-format appendix. Separate companion PDFs existed in `documents/`: `Security_Requirements.pdf`, `User_Stories.pdf`, `Use_Case_Flow.pdf`, `Test_Cases.pdf`.

Version	Date	Author / change	Description
2.0	6 November 2026	Updated SRS (this document)	Expanded into a full IEEE-format SRS. Kept the FR/NFR numbering and wording of v1.0 where it matches the code; added FR-11 to FR-14 for features present in the code but absent from v1.0 (batch compression, compression history, performance measurement, output/overwrite protection); made the NFRs measurable; added security objectives and SR-1 to SR-10; added a text use-case diagram and use-case descriptions; added traceability and appendices. Deviations from the initial design are listed in §2.5 and §5.

Numbering note — FR-9 and FR-10

In `documents/SRS_document.pdf` the two items appear in the opposite order: viewing codes / visualising the tree is listed there as FR-9, and invalid/corrupt handling as FR-10. This v2.0 adopts **FR-9 = detect and handle invalid or corrupt files** and **FR-10 = view Huffman codes and visualise the Huffman tree**, so that the error-handling requirement sits with the other high-priority core requirements. The wording of both requirements is otherwise unchanged. If the assessor prefers the v1.0 order, only the two IDs need to be swapped back; no behaviour changes.

Table of contents

- 1 Introduction
- 1.1 Purpose
- 1.2 Scope
- 1.3 Definitions, acronyms and abbreviations
- 1.4 References
- 1.5 Overview
- 2 Overall description
- 2.1 Product perspective
- 2.2 Product functions — summary
- 2.3 User characteristics
- 2.4 Constraints
- 2.5 Assumptions, dependencies and deviations from the initial design
- 3 Specific requirements
- 3.1 Functional requirements (FR-1 to FR-14)
- 3.2 Non-functional requirements (NFR-1 to NFR-8)
- 3.3 Security requirements
- 3.4 Use cases
- 4 Traceability
- 5 Appendices
- Appendix A — Backlog summary
- Appendix B — File format (`.hfc` / `HFC1`)
- Appendix C — Glossary
- Items not in the evidence inspected

1 Introduction

1.1 Purpose

This document specifies what the File Compression Tool must do, and how its behaviour can be checked, so that:

1. a user can compress a file with Huffman coding, decompress it later, and get the original file back byte-for-byte;
2. a user can see *how* the compression worked — frequencies, Huffman codes, the Huffman tree, and statistics;
3. the tool fails safely on invalid, empty, or corrupt input instead of silently producing a wrong result or destroying an existing file;
4. an assessor in a demo or viva can trace every requirement to a backlog item, a code module, and a test case, and can test it from the CLI or the GUI.

The intended readers are the three team members, the mini-project assessor, and any later maintainer. The document is written in plain, precise language for that purpose. It describes *what* the system must do, not the internal code in full — the code is cited only where a requirement needs an exact, checkable definition (for example, the `.hfc` file format in Appendix B).

This v2.0 replaces the v1.0 table-only SRS as the single SRS for the project. The companion documents are not discarded: they are listed as references in §1.4 and their contents are folded into this document, with sources named.

1.2 Scope

The product is a **standalone, local, single-user desktop tool** that compresses and decompresses files using **Huffman coding** in Python.

In scope:

- Selecting a file and compressing it into a `.hfc` file (magic `HFC1`), and decompressing a valid `.hfc` file back to the original file.
- Frequency analysis of byte values 0–255, Huffman-tree construction with a hand-written MinHeap, generation of prefix-free Huffman codes, bitstream encoding, and bit-by-bit decoding.
- Lossless verification: decompression must reproduce the original exactly, checked by CRC32 in the file header and by SHA-256 comparison in the `verify` command.
- Displaying compression statistics (original size, compressed size, compression ratio, space saved, average bits per symbol) and performance (elapsed time, throughput, peak memory where requested).
- Viewing the Huffman code table and a text visualisation of the Huffman tree.
- Compressing several files in one batch operation; recording compression/decompression history; protecting existing output files from accidental overwrite.

- Two interfaces to the same compression code: a command-line interface (`main.py` / `compressor/cli.py`) and a 5-tab Tkinter GUI (`gui.py`).

Out of scope (the tool does not do these, and this SRS does not require them):

- Any other compression algorithm (for example, LZ77, DEFLATE, or arithmetic coding). Huffman coding only.
- Encryption, password protection, or access control for compressed files. The `.hfc` format stores the frequency table in plain form (see Appendix B).
- Network use, cloud storage, file sharing, or sending file contents to any external service.
- Multi-user use, user accounts, or a server.
- Editing a file, viewing a file's contents, or compressing a folder/directory as a single archive. Input must be a single file (batch compression processes several single files, one `.hfc` file each — see FR-11).
- Streaming compression of a file larger than the 4 GB limit in Appendix B, or of a file that cannot be read into memory (see §2.4).

Benefit and limitation, stated honestly. For files with repeated data, Huffman coding reduces size. For very small files it can make the file *larger*, because the 19-byte header plus the frequency table (5 bytes per distinct byte value) outweigh the bits saved in the payload. The verified example is the 10-byte file `sample.txt` containing `AAAABBBCCD`: it compresses to 42 bytes (ratio 0.24:1, space saved −320%). See §3.2, NFR-2, and Appendix B for the calculation. The tool does not hide this: it reports the true sizes and ratio, and the use-case flow (Use_Case_Flow.pdf, Alternate Flow A2) expects the user to be told when no size reduction was achieved. No claim is made in this SRS about compression ratios for larger files beyond the qualitative statement that larger, repetitive files compress well — measured ratios for named larger files are in the Test Plan / Testing Reports: a 90,000-byte repetitive file compresses to 50,914 bytes (43.43% saved, ratio 1.77:1).

1.3 Definitions, acronyms and abbreviations

Term	Meaning in this document
Huffman coding	A lossless compression method that gives shorter binary codes to more frequent symbols and longer codes to rarer symbols, so the total number of bits is reduced when some symbols repeat.
Symbol / byte value	One byte of the input file, as a number from 0 to 255. The tool works on bytes, not on characters in any particular text encoding. A text file is therefore handled as its bytes.
Frequency / frequency table	For each distinct byte value, how many times it occurs in the input file. Example: `AAAABBBCCD` gives A=4, B=3, C=2, D=1.
Huffman tree	A binary tree built from the frequency table by repeatedly combining the two lowest-frequency nodes. Each leaf is one byte value; each internal node holds the combined frequency of its children. The root frequency is the total file size.
Huffman code	The binary string for one byte value, read from the root of the tree to its leaf: left edge = `0`, right edge = `1`. Codes are prefix-free: no code is the start of another code. A file with only one distinct byte value assigns that byte the code `0`.
Prefix-free	

Term	Meaning in this document
	No Huffman code is a prefix (starting part) of another Huffman code. This is what makes decoding unambiguous, bit by bit.
Bitstream / payload	The encoded data: the input bytes replaced by their Huffman codes, packed into whole bytes. The last byte is padded with `0` bits if needed; the number of padding bits (0–7) is stored in the header.
`.hfc` / `HFC1`	The tool's compressed-file format. Extension `.hfc`; the first four bytes are the magic value `HFC1`. Full layout: Appendix B.
CRC32	A 32-bit checksum stored in the `.hfc` header at compression time and checked at decompression time. A mismatch means the restored data is not the original.
SHA-256	A hash used by the `verify` command to compare the original file with the file restored from a `.hfc` file, without relying only on the stored CRC32.
Lossless	Decompression reproduces the original file exactly, byte-for-byte, with no information lost.
Ratio	Compression ratio = original size ÷ compressed size, shown as e.g. `2.50:1`. A ratio above 1 means the file got smaller; below 1 means it got larger.
Space saved (%)	`(original size − compressed size) ÷ original size × 100`. Negative when the compressed file is larger.
Average bits per symbol	Total encoded bits ÷ original size in bytes.
MinHeap	The priority queue used to build the Huffman tree: repeatedly takes the two lowest-frequency nodes. In this project it is hand-written in `compressor/min_heap.py` (no `heapq` library), with deterministic tie-breaking by insertion order.
CLI	Command-line interface (`main.py`, `compressor/cli.py`).
GUI	Graphical user interface (`gui.py`): a 5-tab Tkinter window titled "File Compression Tool | Huffman Coding".
BL	Backlog item in the GitHub Project `File_Compression_Tool_BLs` (BL1–BL20). See Appendix A.
FR / NFR / SR / UC / US / UT	Functional Requirement / Non-Functional Requirement / Security Requirement / Use Case / User Story / Test Case (as numbered in the source documents).
SRS / SAD / Test Plan	Software Requirements Specification / Software Architecture & Design / Test Plan.
SDLC	Software Development Life Cycle.

1.4 References

#	Document / source	What it is used for in this SRS
R1	IEEE Std 830-1998, *IEEE Recommended Practice for Software Requirements Specifications*	The format followed by this document (introduction, overall description, specific requirements, traceability, appendices).
R2		

#	Document / source	What it is used for in this SRS
	`documents/SRS_document.pdf` (v1.0) — in the submitted zip	Source of FR-1 to FR-10 and NFR-1 to NFR-8, kept with the numbering note in the Revision History.
R3	`documents/Security_Requirements.pdf` — in the submitted zip	Source of SR-1 to SR-10, reproduced in §3.3.
R4	`documents/User_Stories.pdf` — in the submitted zip	Source of US-01 to US-10, mapped to FRs in §4.
R5	`documents/Use_Case_Flow.pdf` — in the submitted zip	Source of the main compression flow and Alternate Flows A1–A3, used in §3.4.
R6	`documents/Test_Cases.pdf` — in the submitted zip	Source of UT_01 to UT_10, mapped to FRs in §4. Note: its sample output name is `sample.huff`; the code uses `.hfc` — see §2.5. Its "Actual Result" column is blank ("To be filled after execution").
R7	`README.md` — in the submitted zip	Project description, documentation list, and author (Keerthana Kumaresh).
R8	Source code in the submitted zip: `compressor/*.py`, `gui.py`, `main.py`; tests in `tests/*.py`	The authoritative check for what the tool actually does. Cited by module name in §4. Where code and an earlier document disagree, the code is stated and the difference is noted (§2.5).
R9	`01FAQs_MiniProjectDemo.pdf` (Final Demo and Viva Preparation FAQs)	Requires the updated IEEE-format SRS, SAD, and Test Plan in the repository, a maintained GitHub backlog with story points, and Sprint 1 / Sprint 2 outcome videos. This SRS does not claim the videos or any PR / Actions history — none were in the evidence inspected; check the repository itself.
R10	`02TheCheckList.pdf` (Evaluation checklist)	SRS checklist used for this document: IEEE format; introductory and descriptive sections; FRs and NFRs clearly and measurably stated; at least 1 UML Use Case Diagram; security section with at least 2 objectives and 2 requirements.
R11	GitHub Project `File_Compression_Tool_BLs` — backlog screenshot (`image-105.png`)	Source of BL1–BL20, assignees, sprints, story points, and status (all Done). Reproduced in Appendix A.

1.5 Overview

The rest of this document is organised as follows. Section 2 describes the product as a whole: what it is, what it does in outline, who uses it, and the constraints and assumptions it works under, including where it differs from the initial design. Section 3 is the checkable core: functional requirements FR-1 to FR-14, non-functional requirements NFR-1 to NFR-8, security requirements SR-1 to SR-10 under security objectives, and use cases UC-1 to UC-6 with a text use-case diagram. Section 4 traces each requirement to its backlog item, code module, and test file. The appendices give the full backlog table, the exact `.hfc` file format, and a glossary.

2 Overall description

2.1 Product perspective

The File Compression Tool is a **new, standalone desktop application**, not an add-on to another system and not a replacement for an existing product. It has no server, no database, and no network connection. It reads files from the user's own computer and writes files back to the user's own computer.

It has two interfaces to one compression pipeline:

- **CLI** — `main.py` calls `compressor/cli.py`. Commands: `compress`, `decompress`, `verify`, `history`. The `compress` command accepts `-o` / `--output`, `--overwrite`, `--table`, `--tree`, `--verify`, and `--time`. The `decompress` command accepts `-o` / `--output`, `--overwrite`, and `--time`. `verify` takes an original file and a compressed file. `history` shows the stored operation history, or clears it with `--clear`.
- **GUI** — `py gui.py` opens a Tkinter window titled "File Compression Tool | Huffman Coding" with five tabs: **Compress File**, **Decompress File**, **Huffman Explorer**, **Statistics & Performance**, and **Compression History**.

Both interfaces use the same modules in `compressor/`: frequency analysis, tree construction, code generation, file format, pipeline, decompression, statistics, history, performance, and output protection. The GUI additionally keeps its own in-memory session history and session performance record (see FR-12 and FR-13 — the GUI history is *session-only*, while the CLI/pipeline history is stored as JSON on disk).

A compressed file is self-contained: the `.hfc` file holds the original size, the CRC32 of the original data, the padding, and the full frequency table, followed by the encoded payload (Appendix B). Decompression therefore needs no separate key, table, or original file — it rebuilds the Huffman tree from the frequency table stored in the file. This is FR-6, and it is also why a `.hfc` file is larger than the payload alone, especially for small files (§1.2).

The system does not modify the input file. Compression reads the input and writes a new output file; decompression reads the `.hfc` file and writes a new restored file. The default restored name is the compressed name without `.hfc`, with `_restored` added before the extension (for example, `sample_restored.txt`).

2.2 Product functions — summary

In outline, the product must:

1. **Select and validate a file** — accept a valid, readable, non-empty file; reject a missing file, a directory, an unreadable file, or (by default) an empty file, with a clear error message (FR-1, FR-9).
2. **Analyse it** — count how often each byte value 0–255 occurs, sorted by byte value (FR-2).

3. **Build the Huffman tree and codes** — combine the lowest-frequency nodes using a hand-written MinHeap, with deterministic tie-breaking; assign a prefix-free binary code to each byte value (FR-3, FR-4).
4. **Encode and store it** — pack the Huffman bits into bytes and write a `.hfc` file containing the header, frequency table, and payload (FR-5, FR-6).
5. **Decode it** — read a valid `.hfc` file, rebuild the tree, decode bit by bit, and recreate the original file, checking the frequency total and the CRC32 (FR-7).
6. **Show the result** — original size, compressed size, ratio, space saved, and average bits per symbol; elapsed time and throughput where requested (FR-8, FR-13).
7. **Explain itself** — show the Huffman code table and a text visualisation of the tree (FR-10).
8. **Fail safely** — detect invalid, corrupt, truncated, or incomplete input and report the failure instead of producing an invalid result (FR-9, §3.3).
9. **Work in bulk and remember** — compress several files in one batch, and keep a history of operations (FR-11, FR-12).
10. **Protect existing files** — never overwrite an existing output file unless the user explicitly allows it (FR-14).

The detailed, checkable form of each function is in §3.1.

2.3 User characteristics

There is one actor: **the User**. No administrator, no second user role, and no separate operator role exist in the code or the source documents.

The user is assumed to be able to:

- use a desktop computer, choose a file in a file dialog (GUI) or type a file path (CLI), and choose where an output file should be saved;
- understand the four statistics shown after compression (original size, compressed size, ratio, space saved) at the level explained in §1.3 — no knowledge of Huffman coding is needed to compress or decompress a file;
- read a short error message (for example, "File not found" or "Output file already exists") and choose another file or output name.

The user is *not* assumed to know Huffman coding, binary trees, or the `.hfc` format. The Huffman Explorer tab exists for a user who wants to learn how the compression worked, and for the viva, where a team member must be able to explain it. The tool itself must be usable without that knowledge.

2.4 Constraints

#	Constraint	Source / evidence
C1	**Language and libraries.** Python. The code as inspected imports only the Python standard library (`argparse`, `struct`, `zlib`, `json`, `hashlib`, `time`, `tracemalloc`, `pathlib`, `datetime`, `tkinter`, and similar). No third-party package is imported by	`compressor/*.py`, `gui.py`, `main.py`

#	Constraint	Source / evidence
	the code inspected. No `requirements.txt` was present in the zip inspected.	
C2	**Python version.** The compiled files in the zip are for CPython 3.12 (`cpython-312`). The minimum Python version the tool is claimed to support is not stated in the documents inspected. The code was tested on Python 3.12.3 (Ubuntu 24.04.5 LTS).	Zip contents
C3	**Operating system.** The GUI uses Tkinter and the code uses `pathlib` / `os.path`, which are cross-platform in principle. The operating system actually tested is Ubuntu 24.04.5 LTS (Python 3.12.3). No Windows or macOS run is recorded. The GUI run command given in `gui.py` is `py gui.py` (the Windows Python launcher).	`gui.py`
C4	**File size limit.** A file larger than 4 GB (`MAX_FILE_SIZE = 0xFFFFFFFF` bytes, the largest value that fits in the header's unsigned 32-bit size field — see Appendix B) is rejected at compression time with `FileCreationError: File is too large to compress (limit is 4 GB).`	`compressor/file_format.py`
C5	**Memory.** Compression and decompression read the whole input file into memory (`read_file_bytes`). Memory use therefore grows with file size. The tool does not stream the input file during compression. (The `verify` hash helper does read in 1 MB chunks; that is the exception.) Performance measurement can report peak memory when `--time` is used.	`compressor/file_validation.py`, `compressor/verification.py`, `compressor/performance.py`
C6	**Local operation only.** No network, no server, no database, no user accounts. History is a local JSON file (FR-12). File contents are not transmitted anywhere (SR-9).	`compressor/history.py`, §3.3
C7	**Single file per operation (except batch).** One compress/decompress operation handles one input file and produces one output file. Batch compression (FR-11) handles several input files by running the single-file operation once per file. A directory cannot be selected as input, and a directory cannot be compressed as one archive.	`compressor/file_validation.py`, `compressor/batch_compression.py`
C8	**Empty files.** Empty input is rejected by default (`allow_empty=False` in `validate_input_file`). The decompressor's internal handling of an original size of 0 exists only for completeness of the format checks; a user cannot create such a file through the normal compress path, because compression rejects the empty input first.	`compressor/file_validation.py`, `compressor/decompressor.py`
C9	**Tree display depth.** The text tree renderer (`render_tree`) has a default maximum depth of 8; deeper levels are shown as `...`. This is a display limit, not a limit on the tree itself.	`compressor/tree_visualizer.py`
C10	**History size.** Stored history is capped at 100 entries (`MAX_HISTORY_ENTRIES = 100`); older entries are dropped. The GUI history is separate and session-only (FR-12).	`compressor/history.py`, `gui.py`
C11	**No encryption.** The `.hfc` format does not encrypt the payload or the frequency table. Anyone with the `.hfc` file can	Appendix B

#	Constraint	Source / evidence
	decompress it with this tool. This SRS does not require confidentiality of the compressed contents beyond local file handling (SR-9).	

2.5 Assumptions, dependencies and deviations from the initial design

Assumptions:

- A1. The user has read and write permission for the input file, the output location, and (for stored history) the home directory, where `~/.file_compression_tool/history.json` is written. If permission is missing, the tool reports the failure (FR-9, SR-10) rather than succeeding silently.
- A2. The user supplies files that the operating system can read as bytes. The tool treats every file as bytes; it does not depend on the file being text, and it does not depend on any text encoding.
- A3. The same Huffman-tree construction is used for compression and decompression (sorted insertion, deterministic tie-breaking by insertion order). This is what makes the tree rebuilt at decompression time identical to the tree used at compression time, even though only the frequency table — not the tree itself — is stored in the file.
- A4. The assessor can run the tool from the project root, as the GUI docstring states (`py gui.py`), and can run the CLI through `main.py`.

Dependencies:

- D1. Python 3 with Tkinter, for the GUI. If Tkinter is unavailable, the GUI cannot start; the CLI does not need Tkinter.
- D2. The Python standard library only (see C1). No external service, package download, or licence is required by the code inspected.

Deviations from the initial design — documented here, as the FAQs and checklist require:

- DV1. **The SRS itself.** The initial `documents/` folder contains five separate PDFs (SRS, Security Requirements, Test Cases, Use Case Flow, User Stories), not a single IEEE-format SRS. This v2.0 document is the single updated IEEE SRS that brings them together. The separate PDFs remain as source references (§1.4).
- DV2. **Two interfaces, not one.** The early documents describe file selection and compression in interface-neutral terms. The implemented product has *both* a CLI and a 5-tab Tkinter GUI. This SRS specifies both, and notes where they differ (notably history: FR-12).
- DV3. **File extension `.hfc`, not `.huff`.** `Test_Cases.pdf` (UT_07, UT_08) uses `sample.huff` as the sample compressed file. The implemented format uses the extension `.hfc` (`EXTENSION = ".hfc"` in `compressor/file_format.py`), the magic `HFC1`, and the GUI file dialog offers `*.hfc`. This SRS uses `.hfc` throughout. The old test-case sample name should be read as `.hfc` when the test cases are updated (see the Test Plan).
- DV4. **Extra functions beyond v1.0.** Batch compression, compression history, performance measurement, and output/overwrite protection exist in the code (`compressor/`

`batch_compression.py`, `history.py`, `performance.py`, `output_protection.py`) and in the backlog (BL18, BL19, BL16, BL14), but have no FR in v1.0. They are added here as FR-11 to FR-14 rather than left undocumented.

- DV5. **History is two things, not one.** The pipeline/CLI history is persistent JSON on disk; the GUI History tab is an in-memory list for the current session only, cleared when the application closes (`gui.py`: "this history is held in memory and is cleared when the application closes"). The v1.0 documents do not make this distinction. FR-12 states it explicitly.
- DV6. **No size-reduction warning as a separate message.** The Use Case Flow (Alternate Flow A2) says the system informs the user when compression does not reduce file size. The code inspected reports the true statistics (including a negative space-saved percentage) in the CLI and GUI, but no separate "no significant reduction" warning dialog or message was found in the code inspected. A search of the code found no such message anywhere in the product. The requirement in this SRS is therefore stated as the statistics being shown truthfully (FR-8), and the A2 flow is recorded in §3.4 with that qualification.
- DV7. **FR-9 / FR-10 order.** See the Revision History note above.

---

3 Specific requirements

3.1 Functional requirements (FR-1 to FR-14)

Priorities are High / Medium / Low, as in v1.0 for FR-1 to FR-10. Priorities for the new FR-11 to FR-14 are assigned in this revision and are stated in each table: FR-11 Low (matching User Story US-10, Low), FR-12 Medium, FR-13 Medium, FR-14 High (matching Security Requirement SR-3, High).

Each table gives the acceptance criterion — the observable check an assessor can run — and the source backlog item(s). The module and test-file trace for every FR is in §4.

FR-1 — Select a file to compress

Field	Content
ID	FR-1
Description	The system shall allow the user to select a file that needs to be compressed. In the GUI this is a file chooser ("Choose a file to compress"); in the CLI it is the input file argument to the `compress` command.
Priority	High
Acceptance criteria	When a valid file is selected, the system accepts it as the input for compression and shows its name (and size, in the GUI Explorer / Compress tab). When no file is chosen, the GUI warns "Choose a file first" and does nothing.
Source	BL1 — File Selection; US-01; UT_01

FR-2 — Calculate byte frequencies

Field	Content
ID	FR-2
Description	The system shall read the input file and calculate the frequency of each byte (value 0–255) in it. Frequencies are returned sorted by byte value.
Priority	High
Acceptance criteria	For a selected file, the system produces frequency information for every byte value present. For the test file `AAAABBBCCD` (10 bytes) the frequencies are A=4, B=3, C=2, D=1. The sum of all frequencies equals the original file size.
Source	BL3 — Frequency Analysis; US-02, US-03; UT_03

FR-3 — Construct a Huffman tree

Field	Content
ID	FR-3
Description	The system shall construct a Huffman tree based on the calculated byte frequencies, using a hand-written MinHeap (no `heapq`), repeatedly combining the two lowest-frequency nodes. Insertion is in sorted byte order, and equal frequencies are taken out in insertion order, so the tree is built identically every time for the same frequencies. An empty frequency table gives no tree (`None`).
Priority	High
Acceptance criteria	A Huffman tree is generated from the frequency data for the selected input file. For `AAAABBBCCD` the root frequency is 10 (the total file size). Building the tree twice from the same frequencies gives the same tree.
Source	BL4 — Min Heap; BL5 — Huffman Tree; US-02; UT_04

FR-4 — Generate Huffman codes

Field	Content
ID	FR-4
Description	The system shall generate variable-length binary Huffman codes for each byte using the constructed tree. The left edge represents `0` and the right edge represents `1`. The codes are prefix-free. A tree with only one symbol assigns that symbol the code `0`. An empty tree gives no codes.
Priority	High
Acceptance criteria	Each byte represented in the tree is assigned a corresponding Huffman code, and no generated code is a prefix of another generated code. For `AAAABBBCCD` the codes are A=`0`, B=`10`, D=`110`, C=`111`.
Source	BL6 — Huffman Codes; US-02, US-03; UT_05

FR-5 — Encode the file and create a compressed file

Field	Content
ID	FR-5
Description	The system shall encode the original file using the generated Huffman codes and create a compressed file. The encoded bits are packed into bytes; the final byte is padded with `0` bits if needed, and the padding count (0–7) is stored in the header. The default output name is the input name plus `.hfc`.
Priority	High
Acceptance criteria	After compression, a compressed output file is created successfully at the selected location and is readable by the application for decompression. For `AAAABBBCCD`, the payload is 19 bits packed into 3 bytes.
Source	BL7 — Bitstream Encoding; BL8 — Compressed File Creation; US-02; UT_06, UT_07

FR-6 — Store the information needed for decompression

Field	Content
ID	FR-6
Description	The system shall store the compressed data together with the information required to reconstruct the Huffman tree during decompression — namely the frequency table — in the `.hfc` file. The file format is defined in Appendix B: a 19-byte header (magic `HFC1`, original size, CRC32, padding, frequency count), followed by one 5-byte entry per distinct byte value (symbol byte + count, sorted by symbol), followed by the payload. The tree itself is not stored; it is rebuilt from the frequency table.
Priority	High
Acceptance criteria	A valid compressed file contains sufficient information for the system to reconstruct the required Huffman tree. Decompressing a file produced by the tool, without any other file, reproduces the original (see FR-7).
Source	BL8 — Compressed File Creation; US-04; UT_06, UT_07

FR-7 — Decode a valid compressed file and recreate the original file

Field	Content
ID	FR-7
Description	The system shall decode a valid compressed file using the Huffman tree rebuilt from its stored frequency table, and recreate the original file. Decoding walks the tree bit by bit. The special cases are handled as in the code: a file with a single distinct byte is recreated by repeating that byte to the original size (with no payload expected); the frequency total must equal the original size; and the CRC32 of the restored data must match the CRC32 stored in the header.
Priority	High
Acceptance criteria	When a valid compressed file is decompressed, the original file is recreated byte-for-byte. The default restored name is the compressed name without `.hfc`, with `_restored` added before the extension.
Source	BL9 — File Decompression; US-04, US-05; UT_08, UT_09

FR-8 — Display compression statistics

Field	Content
ID	FR-8
Description	The system shall display the original file size, compressed file size, compression ratio, and percentage of space saved after compression. It shall also be able to show space saved in bytes and average bits per symbol, as calculated by `compressor/compression_stats.py`: ratio = original ÷ compressed; space saved % = (original − compressed) ÷ original × 100; average bits per symbol = total encoded bits ÷ original size. For an original size of 0, the ratio is shown as "N/A" and space saved as 0.
Priority	Medium

Field	Content
Acceptance criteria	After compression, all four values (original size, compressed size, compression ratio, percentage of space saved) are displayed to the user, in the CLI output and in the GUI Compress tab / Statistics & Performance tab. For the worked example in UT_10 (original 100 KB, compressed 60 KB), the displayed ratio is 0.60 as a fraction (shown as `1.67:1` in the tool's original÷compressed form — see note below) and space saved is 40%.
Source	BL11 — Compression Statistics; US-06; UT_10

> **Note on the UT_10 numbers.** `Test_Cases.pdf` UT_10 states a 100 KB original and a 60 KB compressed file, a "compression ratio (0.60)", and "space saved (40%)". The tool's own definition is ratio = original ÷ compressed, which for those sizes is 100 ÷ 60 = 1.67, shown as `1.67:1`. The 0.60 in UT_10 is compressed ÷ original. Both describe the same pair of sizes; the 40% space saved is the same either way. This SRS uses the tool's definition (original ÷ compressed, Appendix B / §1.3) and records the difference here so the test case can be corrected to `1.67:1` when it is updated.

FR-9 — Detect and handle invalid, corrupt, or failed operations

Field	Content
ID	FR-9
Description	The system shall detect and handle invalid files, corrupted compressed files, and unsuccessful compression or decompression operations. It shall report the failure instead of producing an invalid result. This covers: a missing, unreadable, or empty input file, or a directory given as input (BL2); a `.hfc` file with a wrong magic value, invalid padding (> 7), an incomplete or duplicate-symbol frequency table, a zero frequency, a frequency total that does not match the original size, a truncated or over-long payload, or a CRC32 mismatch (BL15, decompressor checks); and a failed write (SR-10). Failures are reported with a clear message and a non-zero exit code in the CLI (see §3.3), and with an error dialog in the GUI. No traceback is shown to the user for an expected failure.
Priority	High
Acceptance criteria	When an invalid or corrupted input is encountered, the system reports the failure instead of producing an invalid result: no invalid output file is presented as a success, and a partial output is not left as if it were complete. For a missing file, the message is of the form "File not found" / "Invalid file", and compression does not start (UT_02).
Source	BL2 — File Validation; BL15 — Corrupted File Detection; BL17 — Error Handling; US-09; UT_02

FR-10 — View Huffman codes and visualise the Huffman tree

Field	Content
ID	FR-10
Description	The system shall allow the user to view the generated Huffman codes and visualise the Huffman tree. In the CLI this is `compress --table` (code table) and `compress --tree` (text tree). In the GUI it is the Huffman Explorer tab, which shows the file name, file size, distinct byte count, the code table (byte, frequency, Huffman code, sorted by byte value), and

Field	Content
	the tree rendered as indented text. The tree renderer has a default maximum depth of 8; deeper levels are shown as `...`. Exploring a file creates no output file.
Priority	Low
Acceptance criteria	The system displays the generated codes and a representation of the Huffman tree for the selected file, with the symbol and frequency shown for each leaf, and the codes shown match the codes used for encoding.
Source	BL12 — Huffman Tree Visualization; BL13 — Huffman Code Table; US-03, US-07; UT_05

FR-11 — Compress multiple files in one batch

Field	Content
ID	FR-11
Description	The system shall allow the user to compress multiple files together in one batch operation. Each file is validated, compressed by the same single-file steps as FR-2 to FR-6, and written to its own `.hfc` file (in the same folder as the input, or in a chosen output folder). At least one file must be selected; an empty selection is rejected. The result for each file (input path, output path, original size, compressed size) is returned.
Priority	Low
Acceptance criteria	The user can select multiple valid files; all are processed successfully, each to its own `.hfc` file; and each file's data is preserved (each `.hfc` file decompresses to its own original, per FR-7). A file that fails validation is rejected before its compression starts.
Source	BL18 — Batch Compression; US-10

FR-12 — Keep a compression history

Field	Content
ID	FR-12
Description	The system shall record compression and decompression operations in a history that the user can view, and shall allow the history to be cleared. There are two histories, and they are different: (a) the **stored history** — the CLI/pipeline records each operation as a JSON entry (timestamp in UTC, operation, input file, output file, sizes) in `~/.file_compression_tool/history.json`, keeping at most 100 entries and dropping older ones; a damaged or missing history file is treated as an empty history, not as a crash; the CLI `history` command lists entries and `history --clear` empties the file; (b) the **GUI session history** — the GUI History tab lists only the operations completed during the current application session, held in memory, and cleared when the application closes; its "Clear Session History" button clears only that session list, after confirmation.
Priority	Medium
Acceptance criteria	After a compression or decompression through the CLI/pipeline, an entry appears in the stored history (viewable with the `history` command) with the correct input and output file names. After the 100-entry limit is reached, only the most recent 100 entries are kept. In the GUI, operations in the current session appear in the History tab, and closing the application clears that list.

Field	Content
Source	BL19 — Compression History

FR-13 — Measure performance

Field	Content
ID	FR-13
Description	The system shall measure and display the performance of a compression or decompression operation: elapsed time in seconds, throughput, and — when memory tracking is requested — peak memory. In the CLI this is the `--time` flag on `compress` / `decompress`, which prints time, speed in MB/s, and peak memory where tracked. In the GUI the Statistics & Performance tab shows the most recent operation's elapsed time (to 6 decimal places), input and output sizes, and input throughput in bytes/second, with its compression statistics where the operation was a compression. The GUI states that this is a single-operation timing, not a controlled benchmark, and that results depend on file contents, file size, and the computer.
Priority	Medium
Acceptance criteria	For an operation run with timing requested, the elapsed time and throughput are displayed. The displayed time is the measured time for that operation on that computer; no fixed speed is claimed in this SRS (see NFR-1 for the measurable form of the requirement).
Source	BL16 — Performance Measurement; US-06 ("compression time may also be shown")

FR-14 — Protect existing output files from being overwritten

Field	Content
ID	FR-14
Description	The system shall prevent accidental overwriting of existing files during compression and decompression. By default, if the output file already exists, the operation is refused and the existing file is not modified. Overwriting is allowed only when explicitly requested: in the CLI, with the `--overwrite` flag; in the GUI, there is no overwrite option — the GUI writes with exclusive creation (`xb`) and reports "Output already exists — choose a different output name. The existing file was not changed." The system shall also refuse an output path that is a directory, or whose parent directory does not exist, and shall refuse an output that is the same file as the input (GUI: "The compressed output cannot replace the original file."). Where the check and the write are separated, exclusive creation is used so a file that appears between the check and the write is still not replaced.
Priority	High
Acceptance criteria	If the specified output file already exists and overwrite was not explicitly allowed, the system rejects the operation without modifying the existing file. If `--overwrite` is given in the CLI, the file is replaced.
Source	BL14 — Output & Overwrite Protection; US-08; SR-3

3.2 Non-functional requirements (NFR-1 to NFR-8)

Each NFR is stated so that it can be checked by measurement or by inspection. Where the source documents give no numeric threshold, this SRS states the measurable *check* (what is measured and how) rather than inventing a number and presenting it as a measured result.

ID	Requirement (measurable)	Priority	How it is checked / acceptance criterion
NFR-1 — Performance	Compression and decompression shall complete within a reasonable amount of time for supported file sizes, and the time taken shall be measured and reported for every operation where timing is requested.	High	Elapsed time is reported in seconds (CLI `--time`: 3 decimal places with MB/s throughput; GUI: 6 decimal places with bytes/second throughput). Acceptance: the operation completes and the measured time is displayed; the tool does not hang or crash on a supported file. No maximum-time threshold for a named file size is stated in the source documents, and no timing result is claimed in this SRS. Measured times are in the Testing Reports (90,000 bytes in 0.107 s; 10 bytes in 0.007 s).
NFR-2 — Compression effectiveness	For files that are suitable for Huffman compression, the compressed output shall be smaller than the original, particularly for files containing repeated data. The effectiveness is measured by the ratio and the space-saved percentage in FR-8.	High	For a repetitive file, ratio > 1 and space saved % > 0. For the 10-byte `AAAABBBCCD` file the expected result is the opposite — 42 bytes, ratio 0.24:1, space saved −320% — because the 19-byte header plus the 20-byte frequency table outweigh the 3-byte payload; this is accepted and must be reported truthfully, not hidden. A minimum ratio for a named larger repetitive file is not stated in the source documents. The measured ratio for a 90,000-byte repetitive file is 1.77:1 (43.43% saved) — see Testing Reports.
NFR-3 — Correctness / lossless recovery	The system shall consistently decompress a valid compressed file back to the original data without loss.	High	Repeated decompression of valid compressed files reproduces the original data without loss. Checked by FR-7, by the CRC32 in the header, and by the `verify` command (SHA-256 comparison). Acceptance: restored bytes are identical to the original bytes (0 differing bytes) for every valid `.hfc` file tested.
NFR-4 — Correctness of codes and compressed data	Huffman codes and the resulting compressed data shall be generated correctly, without altering the original information, and shall support accurate decompression.	High	Codes are prefix-free (FR-4); the frequency total equals the original size; the payload length is exactly the length implied by the frequencies, codes, and padding (checked by the corruption checks in FR-9); and decompression reproduces the original (NFR-3). Acceptance: the checks in FR-9 pass for a valid file, and a single changed byte in a `.hfc` file is detected (see SR-2 / SR-4).
NFR-5 — Usability	The interface shall be simple enough for a user to select, compress, and decompress files easily,	Medium	A user can complete file selection, compression, and decompression through the interface without unnecessary steps: in the GUI, Compress = choose file → choose destination → Compress; Decompress = choose `.hfc` file → choose destination →

ID	Requirement (measurable)	Priority	How it is checked / acceptance criterion
	without unnecessary steps.		Decompress. Acceptance: the task can be completed from the GUI or CLI by following the on-screen steps alone, with file name and size shown at selection. No usability test result is claimed in this SRS — no measured usability result was in the evidence inspected.
NFR-6 — Scalability / file sizes	The tool shall be capable of handling files of different sizes without major performance issues, within the stated limit.	Medium	The system successfully processes files of different supported sizes. The hard limit is 4 GB (C4); files above it are rejected with a clear message. Acceptance: small, medium, and larger files within the limit compress and decompress correctly (NFR-3 holds for each). "Without major performance issues" is measured by NFR-1 (time reported for each size); a size-by-size time table is not in the evidence inspected; the two measured sizes (10 bytes and 90,000 bytes) are in the Testing Reports.
NFR-7 — Modularity	The implementation shall be modular, with separate components for frequency analysis, tree construction, encoding, decoding, and file handling.	Medium	The implementation contains logically separated modules for the major operations. Acceptance by inspection: `frequency.py`, `min_heap.py` + `huffman_tree.py`, `huffman_codes.py`, `file_format.py` (encoding / file creation), `decompressor.py` (decoding), `file_validation.py` / `output_protection.py` (file handling), `compression_stats.py`, `history.py`, `performance.py`, `verification.py`, `corruption.py`, and `pipeline.py` (integration) exist as separate modules, and the CLI (`cli.py`) and GUI (`gui.py`) both use them rather than duplicating the algorithm.
NFR-8 — Portability	The application shall be executable on commonly used systems without requiring major changes to the implementation.	Low	The application can be executed on supported commonly used systems with minimal or no implementation changes. Acceptance: the code as inspected uses only the Python standard library and `pathlib` / `os.path` for file paths (C1). The system actually tested is Ubuntu 24.04.5 LTS with Python 3.12.3 (C2, C3); no Windows/macOS portability test result is claimed in this SRS.

3.3 Security requirements

Security objectives

ID	Objective	Why it matters for this tool
SO-1		

ID	Objective	Why it matters for this tool
	Protect the user's existing files from accidental loss.	Compression creates new files next to files the user already has. A wrong output name must never silently destroy an existing file.
SO-2	**Detect corrupt or invalid input before it can produce a wrong result.**	A damaged `.hfc` file, or a file that is not a `.hfc` file at all, must be rejected with a clear message — never decoded into a plausible-looking but wrong file.
SO-3	**Preserve the original data exactly.**	Compression is only useful if the original can be recovered byte-for-byte. Integrity is checked at decompression time, not assumed.
SO-4	**Fail safely, privately, and locally.**	Errors must be reported in plain language without exposing internal details, and file contents must never leave the user's computer.

These are the "at least 2 objectives" required by the checklist; four are given so that every SR below has a named objective. No objective claims encryption, access control, or authentication — the tool provides none of those, and this SRS does not require them (see C11).

Security requirements SR-1 to SR-10

The wording below is kept from `documents/Security_Requirements.pdf`, with the code module that implements each requirement added in the "Implemented by" column. Priorities are as in that document.

ID	Requirement	Priority	Acceptance criteria (from Security_Requirements.pdf)	Objective	Implemented by
SR-1	The system shall validate the selected input file before beginning compression or decompression.	High	When a valid and accessible file is selected, the system accepts it. If the file does not exist or cannot be opened, the system displays an appropriate error message.	SO-2, SO-4	`file_validation.py` (`validate_input_file`); FR-1, FR-9
SR-2	The system shall validate compressed files before decompressing them to ensure that the input follows the expected file format.	High	Valid compressed files are processed. Files with invalid headers, malformed data, or unexpected formats are rejected safely.	SO-2	`decompressor.py` (`parse_compressed`); `corruption.py`; FR-7, FR-9
SR-3	The system shall prevent accidental overwriting of existing files during compression	High	If the specified output file already exists, the system requests confirmation or rejects the operation without modifying the existing file.	SO-1	`output_protection.py`; FR-14

ID	Requirement	Priority	Acceptance criteria (from Security_Requirements.pdf)	Objective	Implemented by
	and decompression.				
SR-4	The system shall preserve the integrity of the original file during compression.	High	After compression and subsequent decompression, the recovered file matches the original file byte-for-byte for all supported inputs.	SO-3	`file_format.py` (CRC32); `decompressor.py`; `verification.py`; FR-7, NFR-3
SR-5	The system shall handle empty files, corrupted input, and unexpected end-of-file conditions without crashing.	High	Empty, corrupted, or incomplete input files are handled gracefully, with appropriate error messages and no abnormal termination.	SO-2, SO-4	`file_validation.py`; `decompressor.py`; `error_handler.py`; FR-9
SR-6	The system shall validate file sizes and input data to reduce excessive memory consumption during compression and decompression.	Medium	Inputs exceeding configured processing limits are handled gracefully without crashing or corrupting existing files.	SO-1, SO-2	`file_format.py` (`MAX_FILE_SIZE` = 4 GB); FR-9; C4, C5
SR-7	The system shall restrict file operations to the input and output paths specified by the user and shall not modify unrelated files.	High	The system reads the specified input and writes to the intended output location without modifying unrelated files.	SO-1	`pipeline.py`; `output_protection.py`; FR-1, FR-14
SR-8	The system shall display appropriate error messages without exposing unnecessary internal implementation details.	Medium	File operation failures produce meaningful error messages without exposing memory contents, internal data structures, or sensitive system information.	SO-4	`error_handler.py` (`format_error`: `Error: <message>`; expected failures return an exit code, not a traceback); FR-9

ID	Requirement	Priority	Acceptance criteria (from Security_Requirements.pdf)	Objective	Implemented by
SR-9	The system shall not transmit user file contents to external services unless explicitly required and authorized.	High	When operating as a standalone application, file processing occurs locally, and no file contents are transmitted to external services.	SO-4	Whole product: local-only, no network code in the modules inspected; C6
SR-10	The system shall safely handle file-processing errors and failed write operations.	High	If the system cannot write the output, it reports the failure and does not present an incomplete file as a successful result.	SO-1, SO-4	`file_format.py` (`write_bytes` removes a partial output on write failure); `output_protection.py`; FR-9, FR-14

Notes and qualifications (read before the viva):

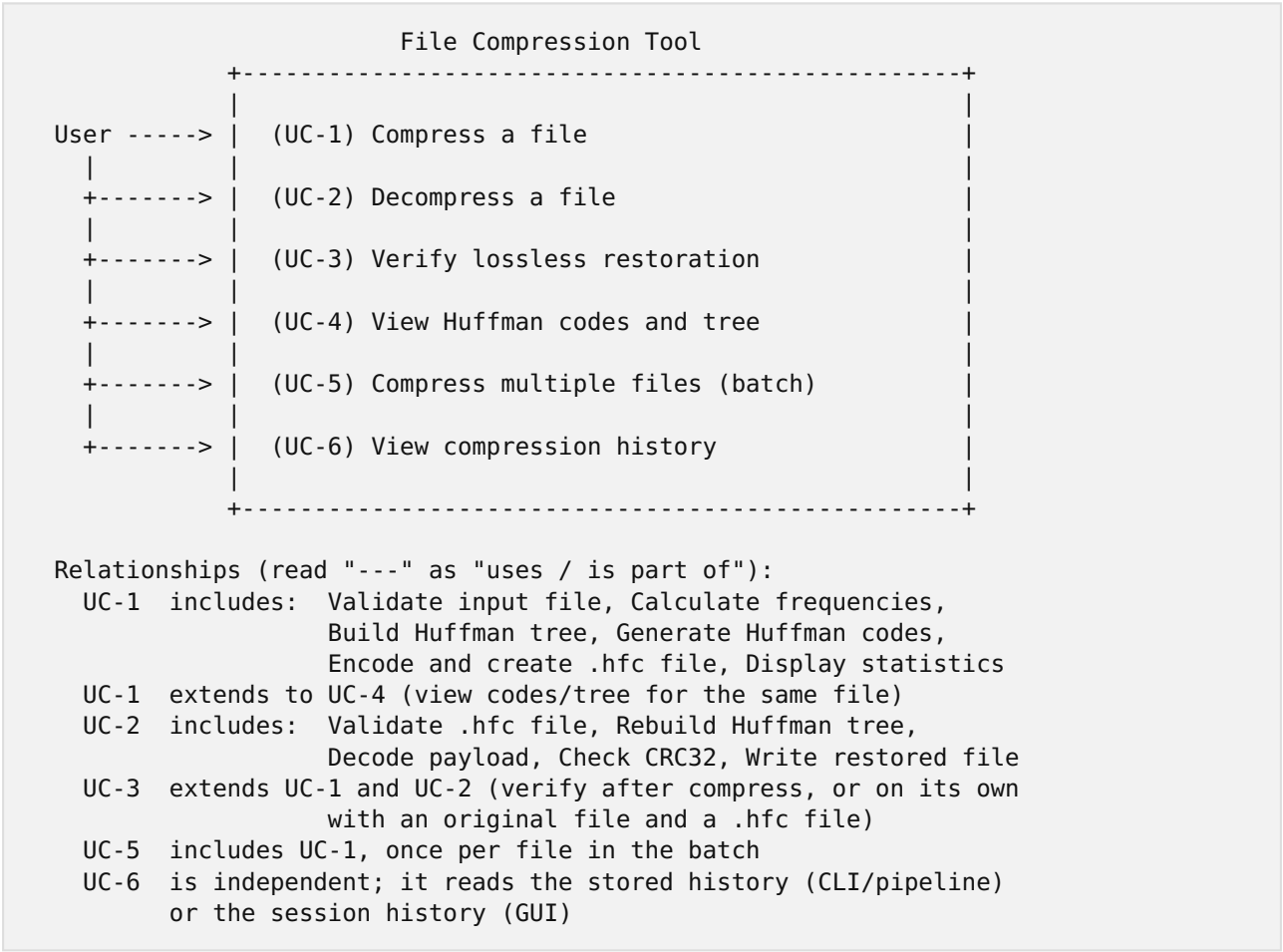
- **SR-3, "requests confirmation or rejects".** The code implements the *reject* half: the CLI refuses an existing output unless `--overwrite` is given, and the GUI refuses it outright (exclusive creation, `xb`). No confirmation dialog that then overwrites was found in the code inspected. The requirement is therefore met by rejection, and the GUI's file-save dialog is not relied on as the protection. If the team wants a confirmation flow as well, that is a change to the code, not a change to this SRS.
- **SR-5, "empty files".** Empty input is rejected before compression (C8). The requirement is met by graceful rejection, not by compressing an empty file.
- **SR-6, "configured processing limits".** The only configured limit in the code is the 4 GB maximum file size. There is no separate configurable memory limit in the code inspected. The requirement is met for file size; no separate configurable memory limit exists in the code inspected.
- **SR-8, exit codes.** The CLI returns distinct exit codes for expected failures (`error_handler.py`): file validation = 2, file creation = 3, corrupt file = 4, output already exists = 5, verification failure = 6, other compression error = 1, out of memory = 7, cancelled by the user = 130. Success = 0.
- **What security does *not* cover.** The tool does not encrypt files, does not authenticate users, and does not protect a `.hfc` file from being read by anyone who has the file. Those are not required by SR-1 to SR-10 and are not claimed here.

3.4 Use cases

Use Case Diagram

The diagram below is a **text (ASCII) diagram**. It is the UML Use Case Diagram required by the checklist, drawn in text so that it is readable in this Markdown file. **No image file of this diagram**

is included with this SRS, and none is claimed to exist. The actor is the User. The system boundary is the File Compression Tool.



The main flow below follows `Use_Case_Flow.pdf`. Its Alternate Flows A1–A3 are given under UC-1 and apply, with the qualifications noted, to the other use cases as well.

UC-1 — Compress a file

Field	Content
Actor	User
Goal	Reduce a file's size by Huffman coding and store the result as a `.hfc` file.
Preconditions	The tool is open (GUI) or the CLI is available. The user has a valid, readable, non-empty file.
Main flow	1. The user opens the File Compression Tool. 2. The user selects a file to compress. 3. The system reads the selected file. 4. The system calculates the frequency of each byte in the file. 5. The system constructs the Huffman tree from the frequencies. 6. The system generates Huffman codes for each byte. 7. The system encodes the original file using the codes. 8. The system creates and stores the compressed (`.hfc`) file. 9. The system calculates the original size, compressed size, compression ratio, and percentage of space saved. 10. The system displays the compression results to the user. 11. The user can view the generated Huffman codes / tree if required (UC-4). 12. The compression process is completed successfully.

Field	Content
Alternate flow A1 — Invalid file	1. The user selects a file. 2. The system attempts to read the file. 3. The file is invalid, unavailable, or cannot be accessed. 4. The system displays an appropriate error message. 5. The user selects another file. 6. The system returns to Step 2 of the Main Flow.
Alternate flow A2 — Compression does not reduce file size	1. The system completes Huffman encoding. 2. The resulting compressed file is equal to or larger than the original file. 3. The system informs the user that no significant size reduction was achieved. 4. The system still provides the compression statistics. 5. The user can choose whether to keep the compressed file or cancel the operation. *Qualification: see DV6 in §2.5 — the statistics (including a negative space-saved percentage) are shown in the code inspected; a search of the code found no separate "no significant reduction" message.*
Alternate flow A3 — Insufficient storage	1. The system completes the compression process and attempts to save the compressed file. 2. The system detects that there is insufficient storage space at the selected output location. 3. The system informs the user that the compressed file cannot be saved. 4. The user selects another output location or cancels the operation. 5. If another location is selected, the system attempts to save the compressed file at the new location. *Qualification: the code reports a failed write and does not present an incomplete file as a success (SR-10); automatic retry at a new location is a user action, not a system retry.*
Postconditions	A `.hfc` file exists at the chosen location (or, in A2, the user has chosen to keep or discard it). The original file is unchanged. Statistics have been displayed. A history entry has been recorded (FR-12).
Requirements	FR-1, FR-2, FR-3, FR-4, FR-5, FR-6, FR-8, FR-9, FR-14; NFR-1, NFR-2, NFR-3, NFR-4; SR-1, SR-3, SR-6, SR-7, SR-10

UC-2 — Decompress a file

Field	Content
Actor	User
Goal	Recover the original file from a valid `.hfc` file.
Preconditions	The user has a valid `.hfc` file produced by this tool.
Main flow	1. The user selects the compressed (`.hfc`) file. 2. The system validates the file format (magic, padding, frequency table). 3. The system rebuilds the Huffman tree from the stored frequency table. 4. The system decodes the payload bit by bit. 5. The system checks that the frequency total matches the original size and that the CRC32 matches. 6. The system writes the restored file to the chosen location. 7. The system reports the restored file and its size.
Alternate flow	If the file is invalid or corrupt (wrong magic, bad padding, incomplete/duplicate frequency table, frequency total mismatch, truncated payload, CRC32 mismatch), the system rejects it with a clear error message and writes no restored file (FR-9, SR-2, SR-5). If the output already exists, the operation is refused unless overwrite was explicitly allowed (FR-14).
Postconditions	The restored file is byte-for-byte identical to the original file. The `.hfc` file is unchanged. A history entry has been recorded (FR-12).
Requirements	FR-7, FR-9, FR-14; NFR-3, NFR-4; SR-2, SR-4, SR-5, SR-10

UC-3 — Verify lossless restoration

Field	Content
Actor	User
Goal	Check, independently, that a `.hfc` file restores the original file exactly.
Preconditions	The user has the original file and the `.hfc` file.
Main flow	1. The user runs the `verify` command with the original file and the compressed file (CLI), or runs `compress --verify` at compression time. 2. The system decompresses the `.hfc` file in memory. 3. The system compares the restored bytes with the original file by SHA-256 hash. 4. The system reports "OK: The compressed file restores the original exactly." — or raises a verification error if the restored data differs.
Alternate flow	If the `.hfc` file is corrupt, verification fails with the corruption error (FR-9) rather than a false "OK".
Postconditions	The user knows whether the restoration is exact. No file is written by `verify` itself.
Requirements	FR-7, FR-9; NFR-3, NFR-4; SR-4

UC-4 — View Huffman codes and the Huffman tree

Field	Content
Actor	User
Goal	Understand how the compression was performed for a selected file.
Preconditions	The user has a valid, readable, non-empty file to analyse.
Main flow	1. The user chooses a file to analyse (GUI: Huffman Explorer tab → "Choose File and Explore"; CLI: `compress --table --tree`). 2. The system calculates frequencies, builds the tree, and generates the codes (FR-2 to FR-4). 3. The system displays the file name, file size, and distinct byte count; the code table (byte, frequency, Huffman code); and the tree as indented text, with symbol and frequency at each leaf.
Alternate flow	If the file is invalid or empty, the system reports the error and shows no codes or tree.
Postconditions	The user can see the codes and tree. No output file is created.
Requirements	FR-10, FR-2, FR-3, FR-4; NFR-5

UC-5 — Compress multiple files (batch)

Field	Content
Actor	User
Goal	Compress several files together without repeating the single-file steps for each one by hand.
Preconditions	The user has two or more valid, readable, non-empty files.
Main flow	1. The user selects multiple files (and, optionally, an output folder and whether overwrite is allowed). 2. For each file in turn, the system runs the UC-1 steps (validate → frequencies →

Field	Content
	tree → codes → encode → write `.hfc`). 3. The system returns the result for each file: input path, output path, original size, compressed size.
Alternate flow	If no file is selected, the operation is rejected ("Select at least one file to compress."). If one file fails validation, that file is rejected before its compression starts.
Postconditions	One `.hfc` file exists per input file. Each `.hfc` file decompresses to its own original. The original files are unchanged.
Requirements	FR-11, FR-1 to FR-6, FR-14; US-10

UC-6 — View compression history

Field	Content
Actor	User
Goal	See what compression and decompression operations have been done, and clear the record if wanted.
Preconditions	None (an empty history is a valid result).
Main flow	1. The user asks to view history (CLI: `history` command; GUI: Compression History tab → "Refresh History"). 2. The system lists the recorded operations — timestamp, operation (compression/decompression), input file, output file — most recent last in the stored list, most recent first in the GUI session list. 3. If there is no history, the system says so ("No history available." / "NO OPERATIONS RECORDED") instead of showing an error.
Alternate flow	To clear: CLI `history --clear` empties the stored JSON file; GUI "Clear Session History" asks for confirmation and then clears only the current session's list. A missing or damaged stored history file is treated as empty (FR-12).
Postconditions	The user has seen the history (or the cleared state). Viewing history does not change any compressed or original file.
Requirements	FR-12

---

4 Traceability

4.1 Requirements → backlog → module → test

"Test file" means a test file present in `tests/` in the submitted zip. "UT" means a test case in `documents/Test_Cases.pdf` (UT_01–UT_10). Where no dedicated test file or UT exists for a requirement, that is stated — it is a gap for the Test Plan, not a claim that a test exists.

FR	Backlog (BL)	Code module(s)	Test file(s) in `tests/`	Test case(s) in Test_Cases.pdf
FR-1	BL1	`file_validation.py`; `gui.py` (Compress tab); `cli.py` (input argument)	`test_integration.py` (CLI parser)	UT_01, UT_02
FR-2	BL3	`frequency.py`	*No dedicated `test_frequency.py` in `tests/`*	UT_03
FR-3	BL4, BL5	`min_heap.py`; `huffman_tree.py`	`test_huffman_tree.py`	UT_04
FR-4	BL6	`huffman_codes.py`	`test_huffman_codes.py`	UT_05
FR-5	BL7, BL8	`file_format.py` (`encode_to_bytes`, `build_compressed_bytes`); `pipeline.py`	`test_file_format.py`; `test_integration.py`	UT_06, UT_07
FR-6	BL8	`file_format.py`	`test_file_format.py`	UT_06, UT_07
FR-7	BL9	`decompressor.py`; `pipeline.py`	`test_integration.py` (complete round-trip)	UT_08, UT_09
FR-8	BL11	`compression_stats.py`	`test_compression_stats.py`	UT_10
FR-9	BL2, BL15, BL17	`file_validation.py`; `corruption.py`; `decompressor.py`; `error_handler.py`; `errors.py`	`test_corruption.py`; `test_error_handler.py`	UT_02
FR-10	BL12, BL13	`tree_visualizer.py`; `huffman_code_table.py`; `gui.py` (Huffman Explorer tab)	`test_tree_visualizer.py`; `test_huffman_code_table.py`	UT_05 (codes displayed) — *no UT for the tree visualisation alone*
FR-11	BL18	`batch_compression.py`	`test_batch_compression.py`	*No UT in Test_Cases.pdf*
FR-12	BL19	`history.py`; `pipeline.py`; `gui.py` (History tab, session-only)	`test_history.py`	*No UT in Test_Cases.pdf*

FR	Backlog (BL)	Code module(s)	Test file(s) in `tests/`	Test case(s) in Test_Cases.pdf
FR-13	BL16	`performance.py`; `cli.py` (`--time`); `gui.py` (Statistics & Performance tab)	`test_performance.py`	UT_10 (statistics); US-06 for time — *no UT for timing alone*
FR-14	BL14	`output_protection.py`; `pipeline.py`; `gui.py` (`_write_new_file`, `xb`)	`test_output_protection.py`	*No UT in Test_Cases.pdf* (US-08)

Notes on the test files:

- `tests/` in the zip contains 14 test files (`test_*.py`) and 81 test functions in total. The "Actual Result" / "Test Result" columns in the original `Test_Cases.pdf` were blank ("To be filled after execution"); the execution results are now filled in the Test Plan and Testing Reports delivered with this SRS — all 81 tests passed, 0 failed.
- The NFRs trace as follows: NFR-1 → BL16, `performance.py`, `test_performance.py`; NFR-2 → BL11, `compression_stats.py`, `test_compression_stats.py`; NFR-3 → BL10, `verification.py`, `test_verification.py` + `test_integration.py`; NFR-4 → BL5/BL6, `huffman_tree.py` / `huffman_codes.py`, `test_huffman_tree.py` / `test_huffman_codes.py`; NFR-5 → GUI (`gui.py`) and CLI (`cli.py`), no dedicated usability test file; NFR-6 → `file_format.py` (4 GB limit) + `pipeline.py`, `test_integration.py`; NFR-7 → module structure as a whole (§3.2, NFR-7); NFR-8 → whole codebase (standard library only), no dedicated test file.
- The SRs trace as follows: SR-1 → BL2; SR-2 → BL9/BL15; SR-3 → BL14; SR-4 → BL10; SR-5 → BL2/BL15/BL17; SR-6 → BL8 (size limit); SR-7 → BL20 (integration); SR-8 → BL17; SR-9 → whole product (local-only); SR-10 → BL8/BL14. Full module mapping is in §3.3.

4.2 User stories → requirements

User story	Requirement(s)
US-01 — Select a file to compress	FR-1
US-02 — Compress a file using Huffman coding	FR-2, FR-3, FR-4, FR-5, FR-6
US-03 — Generate and display Huffman codes	FR-2, FR-3, FR-4, FR-10
US-04 — Decompress a compressed file	FR-6, FR-7
US-05 — Lossless compression	FR-7, NFR-3, NFR-4, SR-4
US-06 — View compression statistics	FR-8, FR-13
US-07 — Visualise the Huffman tree	FR-10
US-08 — Choose the output location; overwrite needs confirmation	FR-14, SR-3
US-09 — Meaningful error messages when an operation fails	FR-9, SR-5, SR-8
US-10 — Compress multiple files together	FR-11

5 Appendices

Appendix A — Backlog summary

Source: GitHub Project `File_Compression_Tool_BLs`, screenshot `image-105.png`. All 20 items are shown as **Done**. Story points total **99** (Sprint (pract): 16; Sprint 1: 44; Sprint 2: 39 — see the subtotals below). Assignees are GitHub usernames. No full names are given for `rishi2908` or `khushi309` in the evidence inspected.

A.1 All backlog items

BL	Title	Issue	Assignee	Sprint	Story points	Status
BL1	File Selection	#1	KeerthanaKumaresh1	Sprint (pract)	3	Done
BL2	File Validation	#2	KeerthanaKumaresh1	Sprint 1	3	Done
BL3	Frequency Analysis	#3	KeerthanaKumaresh1	Sprint 1	5	Done
BL4	Min Heap	#4	KeerthanaKumaresh1	Sprint 1	5	Done
BL5	Huffman Tree	#5	rishi2908	Sprint 1	8	Done
BL6	Huffman Codes	#6	rishi2908	Sprint 1	5	Done
BL7	Bitstream Encoding	#7	rishi2908	Sprint (pract)	5	Done
BL8	Compressed File Creation	#8	khushi309	Sprint 1	5	Done
BL9	File Decompression	#10	KeerthanaKumaresh1	Sprint 2	5	Done
BL10	Lossless Verification	#11	KeerthanaKumaresh1	Sprint 2	3	Done
BL11	Compression Statistics	#12	rishi2908	Sprint 2	3	Done
BL12	Huffman Tree Visualization	#13	khushi309	Sprint 1	5	Done
BL13	Huffman Code Table	#14	rishi2908	Sprint 1	3	Done
BL14	Output & Overwrite Protection	#15	rishi2908	Sprint 2	5	Done
BL15	Corrupted File Detection	#16	KeerthanaKumaresh1	Sprint 2	5	Done
BL16	Performance Measurement	#17	khushi309	Sprint 2	5	Done
BL17	Error Handling	#18	khushi309	Sprint 1	5	Done
BL18	Batch Compression	#19	khushi309	Sprint (pract)	8	Done
BL19	Compression History	#20	rishi2908	Sprint 2	5	Done

BL	Title	Issue	Assignee	Sprint	Story points	Status
BL20	Integration & Final Testing	#21	khushi309	Sprint 2	8	Done
Total	**20 items**				**99**	**All Done**

Notes: Issue #9 does not appear in the backlog screenshot inspected; no reason is given in the evidence, and none is invented here. "Sprint (pract)" is the sprint name as shown in the screenshot.

A.2 Totals by sprint and by person

Sprint	Items	Story points
Sprint (pract)	BL1, BL7, BL18 (3 items)	16
Sprint 1	BL2, BL3, BL4, BL5, BL6, BL8, BL12, BL13, BL17 (9 items)	44
Sprint 2	BL9, BL10, BL11, BL14, BL15, BL16, BL19, BL20 (8 items)	39
Total	**20 items**	**99**

Team member (username)	Items	Story points
KeerthanaKumaresh1 (Keerthana Kumaresh, repository owner)	BL1, BL2, BL3, BL4, BL9, BL10, BL15 (7 items)	29
rishi2908	BL5, BL6, BL7, BL11, BL13, BL14, BL19 (7 items)	34
khushi309	BL8, BL12, BL16, BL17, BL18, BL20 (6 items)	36
Total (3 people)	**20 items**	**99**

The team is **3 people**. This SRS does not claim a team of 4.

Appendix B — File format (`.hfc` / `HFC1`)

This appendix defines the format exactly as implemented in `compressor/file_format.py`. All numbers are stored **big-endian**. The format is the reason a small file can grow: the header and frequency table are fixed overhead, paid once per file, whatever the payload size.

B.1 Layout

```
+----------------------+---------------------------+------------------+
| Header (19 bytes)    | Frequency table           | Payload          |
|                      | (5 bytes per distinct     | (Huffman bits,   |
|                      |  byte value)              |  packed into     |
```

```
|                      |                            |  bytes)          |
+----------------------+----------------------------+------------------+
```

B.2 Header — 19 bytes, struct format `>4sQIBH`

Offset	Size	Field	Format	Meaning
0	4 bytes	Magic	`4s`	Always the bytes `HFC1` (`MAGIC = b"HFC1"`). Any other value means "not a valid compressed file".
4	8 bytes	Original size	`Q` (unsigned 64-bit)	Size of the original file in bytes. Also the sum that the frequency table must add up to.
12	4 bytes	CRC32	`I` (unsigned 32-bit)	`zlib.crc32` of the original data, stored at compression time and checked after decoding.
16	1 byte	Padding	`B`	Number of padding `0` bits (0–7) in the last payload byte. A value above 7 is invalid.
17	2 bytes	Frequency count	`H` (unsigned 16-bit)	Number of distinct byte values in the file, i.e. the number of entries in the frequency table.
Total	**19 bytes**		`HEADER_SIZE = 19`	

B.3 Frequency table — 5 bytes per entry, struct format `>BI`

Each entry is one distinct byte value: 1 byte for the symbol (`B`), followed by 4 bytes for its count (`I`, unsigned 32-bit). Entries are stored **sorted by symbol (byte value)**. There is one entry per distinct byte value, so the table is `5 × (number of distinct byte values)` bytes long. A duplicate symbol, or a count of 0, makes the file invalid.

B.4 Payload

The input bytes, each replaced by its Huffman code, with the resulting bits packed into bytes, most significant bit first. If the total number of bits is not a multiple of 8, the last byte is padded with `0` bits to fill it, and the padding count is stored in the header (§B.2). The payload length in bytes is exactly `(total encoded bits + 7) // 8`; a payload of any other length is corrupt (checked in `compressor/corruption.py` and `compressor/decompressor.py`).

B.5 Limits and checks on reading

- **Maximum file size:** `MAX_FILE_SIZE = 0xFFFFFFFF` bytes (4 GB). A larger file is rejected at compression time.
- **Extension:** `.hfc` (`EXTENSION = ".hfc"`).
- On reading, the system checks, in this order: the file is at least 19 bytes long; the magic is `HFC1`; padding is ≤ 7; the frequency table is complete (the file is long enough to hold it); no symbol is repeated and no count is 0; the frequency total equals the original size; the payload decodes to exactly the original size; and the CRC32 of the decoded data matches the header. Any failure raises `CorruptedFileError` and no restored file is produced.

B.6 Worked example — `sample.txt`, `AAAABBBCCD`

This example is taken from `Test_Cases.pdf` (UT_03–UT_05) and was checked against the code: the same input gives the same result.

Step	Result
Input	10 bytes: `AAAABBBCCD`
Frequencies (FR-2)	A=4, B=3, C=2, D=1; total = 10
Huffman codes (FR-4)	A= `0`, B= `10`, D= `110`, C= `111`
Total encoded bits	4×1 + 3×2 + 2×3 + 1×3 = 19 bits
Payload	19 bits packed into 3 bytes (5 padding bits)
Header	19 bytes
Frequency table	4 distinct bytes × 5 bytes = 20 bytes
Compressed file	**19 + 20 + 3 = 42 bytes**
Ratio	10 ÷ 42 = **0.24:1**
Space saved	(10 − 42) ÷ 10 × 100 = **−320%**

The compressed file is larger than the original. That is expected for this input, and it is not an error: the 39 bytes of header + table are the fixed cost of making the file self-contained (FR-6), and they outweigh the 7 bytes saved in the payload (10 bytes of input → 3 bytes of payload). For a larger file with repeated data, the same fixed cost is a small fraction of the file, and the ratio is above 1.

Appendix C — Glossary

See §1.3 for the definitions used in this document. The terms below are the same list, collected here for quick reference in a viva.

- **Average bits per symbol** — total encoded bits ÷ original size in bytes.
- **Batch compression** — compressing several files in one operation (FR-11).
- **Bitstream / payload** — Huffman codes packed into bytes (§B.4).
- **CRC32** — 32-bit checksum in the `.hfc` header.
- **HFC1 / `.hfc`** — the compressed-file format: magic `HFC1`, extension `.hfc`.
- **Huffman code** — the binary string for one byte value; left = `0`, right = `1`; prefix-free; single-symbol code is `0`.
- **Huffman tree** — binary tree built from frequencies; root frequency = total file size.
- **Lossless** — decompression reproduces the original byte-for-byte.
- **MinHeap** — hand-written priority queue used to build the tree; deterministic tie-breaking by insertion order.
- **Padding** — `0` bits added to fill the last payload byte (0–7), recorded in the header.
- **Prefix-free** — no code is the start of another code.
- **Ratio** — original size ÷ compressed size, shown as e.g. `1.67:1`.
- **SHA-256** — hash used by the `verify` command.

- **Space saved (%)** — (original − compressed) ÷ original × 100.
- **Symbol / byte value** — one byte of input, 0–255.

Items not in the evidence inspected

The following were not in the code or documents inspected, so they are not claimed anywhere in this SRS. Check them in the repository itself before the viva; do not fill them from memory.

1. Full names of the team members `rishi2908` and `khushi309`.
2. Date and author of the v1.0 SRS (`documents/SRS_document.pdf` states neither).
3. A maximum-time threshold for a named file size (NFR-1), and a minimum compression ratio for a named larger file (NFR-2) — the SRS states the measurable check instead.
4. A measured usability result for NFR-5, and a size-by-size timing table for NFR-6.
5. Sprint dates, demo/video links, pull-request numbers, and GitHub Actions history. The FAQs and checklist require Sprint 1 (1-minute) and Sprint 2 (2-minute) outcome videos in the repository; their presence or location must be checked in the repository itself.

End of SRS v2.0 — File Compression Tool, 6 November 2026.